Object oriented paradigm

Abstraction with classes

Object oriented programming

- A programming approach that organizes software design around the concept of objects.
  - Each object is the instance of one and only one classe
  - Each object has a unique identity:
    - It has properties
      - expressed by attributes
      - defining its state
    - It has behavior
      - expressed by methods
      - Often mimicking behaviour of real-world objects
  - An object-oriented program is a collection of objects collaborating together to accomplish a given task.

Core concepts in object-oriented programming

- For a language to be considered object-oriented, it must include the following concepts:
  - Identity
  - Modularity
  - Classes
  - Abstraction
  - Encapsulation
  - Inheritance
  - Polymorphism

Object Identity

- This is the property that distinguishes an object from all other ones
  - Even if two objects have the same values for all their attributes, they are still distinct
  - In Java
    - the operator `==` checks for object identity;
    - the method `equals` check for object equality
  - The language implementation of this concept will establish if, at anytime, we work with a reference to an object or a copy of that object

Modularity

- An object-oriented program is intrinsiqually modular since:
  - The whole program is completly built inside the classes
  - a class is the fundemental (reusable) unit having well-defined responsibility

Classes

- A class is a programmatic construct used to define the properties and behaviours of the objects that will derive from it
- The classes also specify how the objects within a program are linked to interact and collaborate in solving a problem
- They form the foundation of other object-oriented concepts

Abstraction

- Abstraction is the principle of hiding details of an implementation in order to better focus on higher-level features
  - A good program structure should be able to provide different levels of abstraction through its different constructs
- The class is the abstraction unit of object-oriented programming
  - Under the object-oriented paradigm, classes are abstraction of real-world entities of the problem domain

Encapsulation

- Encapsulation is the principle by which classes serve as containers for the information that pertains to them
- Encapsulation enables access control to class information while hiding implementation details
- Under the object-oriented paradigm, both procedural and data abstractions are encapsulated inside the classes

Inheritance and class generalization

- Object-oriented programming allows the definition of subclasses and superclasses
- Inheritance is the mechanism by which all subclasses implicitly acquire the elements of their superclasses
  - An inherited method can be redefined
  - Some methods in a superclass may be abstract (i.e., let undefined)
- Generalization versus Specialization:
  - A superclass is a generalization of the properties and behaviors of its subclasses
  - A subclass is a specialization of the superclass, designed to extend, restrict, or optimize its properties and behaviors

Polymorphism, operation and methods

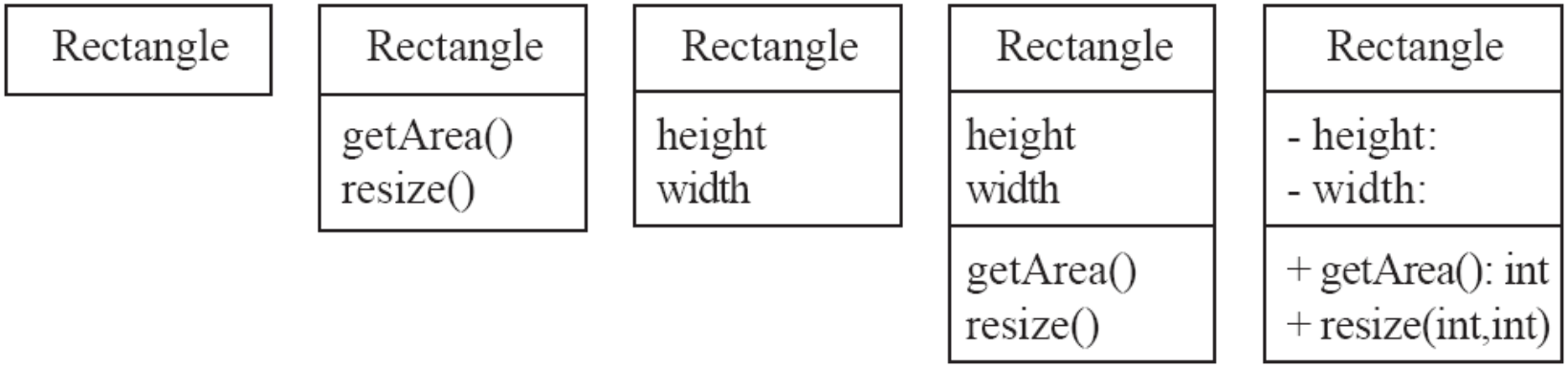
- Operation:
  - An abstract procedural concept corresponding to a specific behaviour
    - e.g. computing the area of a shape
- Method:
  - the implementation of an operation in a program
- Polymorphsim:
  - when the same operation is applied in different ways across different classes
  - it assumes the existence of multiple methods with the same name
  - the method executed by an object depends on the class to which the object belongs
  - this greatly reduces the need for control statements such as `if-else` ou le `switch`

UML and class diagram

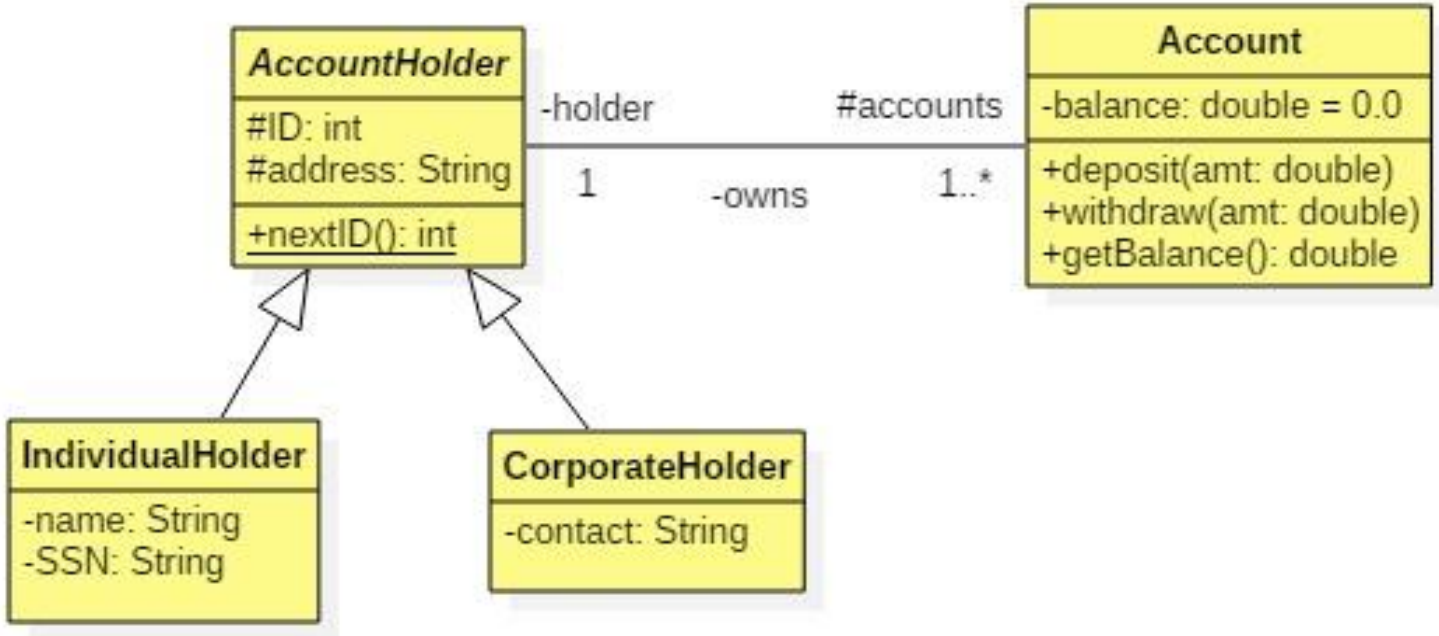
- The « Unified Modelling Language » is a standardized, visual modeling language used to specify, visualize, construct, and document object-oriented software and systems
- UML class diagrams are used to illustrate the classes of a system and their relationships
- The main symbols found in UML diagrams are:
  - classes
  - associations
  - attributes
  - méthods
  - generalizations

Describing a class with UML

- A class is represented by a box showing the name of this cass
  - The diagram can also show the attributes and/or the methods of the class



A simple UML class diagram



https://www.cs.sjsu.edu/~pearce/modules/lectures/uml2/class/index.htm

History of object-oriented languages

- The first object-oriented programming language was Simula-67
  - designed to facilitate the writing of simulation programs.
- In the early 1980s, Smalltalk was developed at Xerox PARC
  - introducing a new syntax, an open-source library, platform independence, garbage collection, and bytecode execution.
- In the late 1980s, C++ was developed by Bjarne Stroustrup
  - combining the benefits of object-oriented programming with the popularity and efficiency of the C language.
  - in 1998, C++ received its first ISO standardization.
- In 1991, engineers at Sun Microsystems launched a project to design a new language (Oak) for embedded programming
  - with the rise of the Internet, a new opportunity emerged for this technology
  - this new language, renamed Java, was officially released in 1995
  - In 2010, Oracle acquired Sun Microsystems and became the owner of the Java platform.